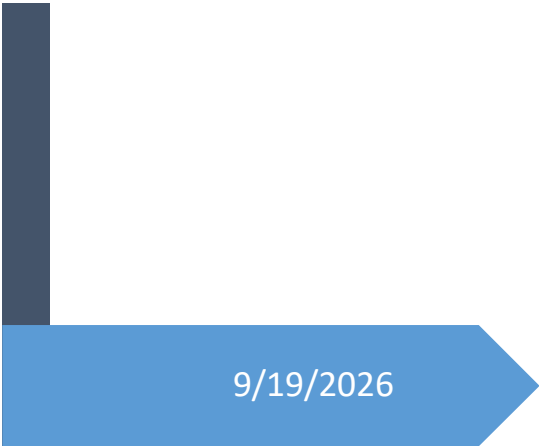
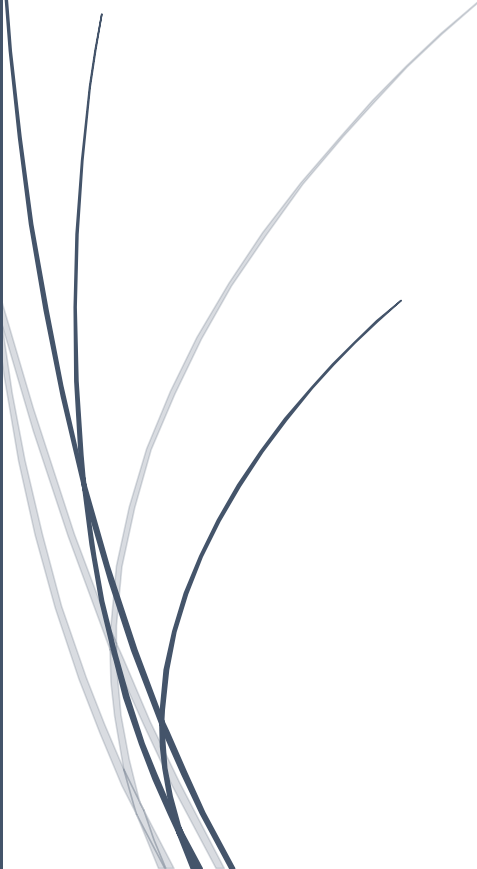


A dark blue vertical bar is on the left. A blue arrow points right from it, containing the date.

9/19/2026

[Lab Task # 1]

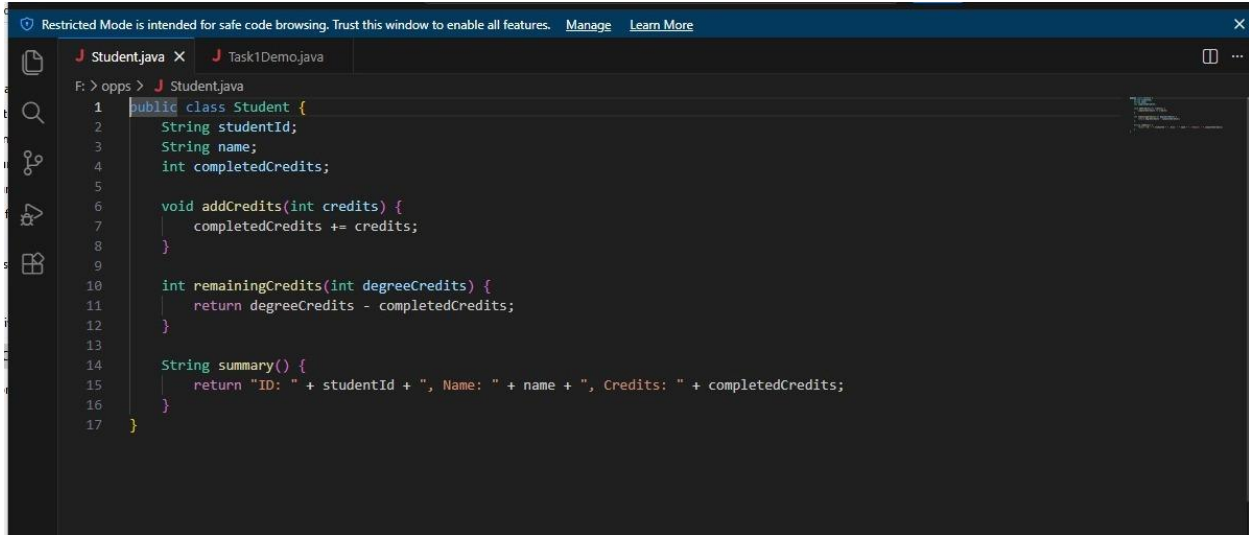
[Course: Object-Oriented Programming (CSC241)]

- 
- Several thin, curved lines in dark blue and light grey originate from the bottom left and curve upwards and to the right.
- **Name:** Hamna Khubaib
 - **Registration:** SP26-BAI-023
 - **Submitted To:** Prof. Muhammad Shahid Bhatti
 - **Date:** September 19, 2026

Task 1 - Build a Class and Prove Independent Object State

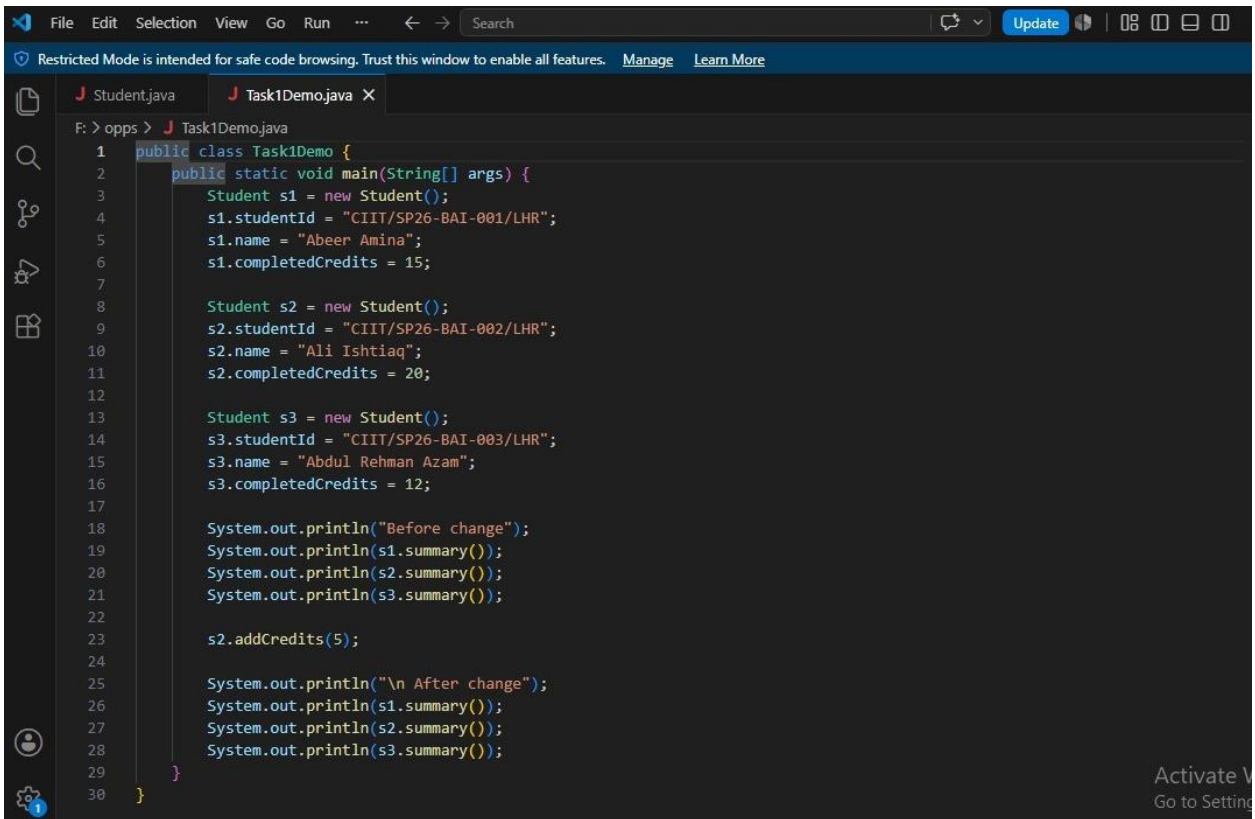
Code Files :

Student .java



```
1 public class Student {
2     String studentId;
3     String name;
4     int completedCredits;
5
6     void addCredits(int credits) {
7         completedCredits += credits;
8     }
9
10    int remainingCredits(int degreeCredits) {
11        return degreeCredits - completedCredits;
12    }
13
14    String summary() {
15        return "ID: " + studentId + ", Name: " + name + ", Credits: " + completedCredits;
16    }
17 }
```

Task1Demo.java



```
1 public class Task1Demo {
2     public static void main(String[] args) {
3         Student s1 = new Student();
4         s1.studentId = "CIIT/SP26-BAI-001/LHR";
5         s1.name = "Abeer Amina";
6         s1.completedCredits = 15;
7
8         Student s2 = new Student();
9         s2.studentId = "CIIT/SP26-BAI-002/LHR";
10        s2.name = "Ali Ishtiaq";
11        s2.completedCredits = 20;
12
13        Student s3 = new Student();
14        s3.studentId = "CIIT/SP26-BAI-003/LHR";
15        s3.name = "Abdul Rehman Azam";
16        s3.completedCredits = 12;
17
18        System.out.println("Before change");
19        System.out.println(s1.summary());
20        System.out.println(s2.summary());
21        System.out.println(s3.summary());
22
23        s2.addCredits(5);
24
25        System.out.println("\n After change");
26        System.out.println(s1.summary());
27        System.out.println(s2.summary());
28        System.out.println(s3.summary());
29    }
30 }
```

Task 1: Command Prompt Output

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

F:\opps>javac Student.java Task1Demo.java

F:\opps>java Task1Demo
Before change
ID: CIIT/SP26-BAI-001/LHR, Name: Abeer Amina, Credits: 15
ID: CIIT/SP26-BAI-002/LHR, Name: Ali Ishtiaq, Credits: 20
ID: CIIT/SP26-BAI-003/LHR, Name: Abdul Rehman Azam, Credits: 12

After change
ID: CIIT/SP26-BAI-001/LHR, Name: Abeer Amina, Credits: 15
ID: CIIT/SP26-BAI-002/LHR, Name: Ali Ishtiaq, Credits: 25
ID: CIIT/SP26-BAI-003/LHR, Name: Abdul Rehman Azam, Credits: 12

F:\opps>
```

Task 1 Concept Check - Answers

1. What is the difference between the Student class and the objects s1, s2, and s3 created from it?

Answer: The `Student` class is a blueprint/template that defines the structure, whereas `s1`, `s2`, and `s3` are specific instances (objects) created in heap memory with their own distinct data.

2. Why does changing `s2.completedCredits` not change the `completedCredits` value stored in `s1` or `s3`?

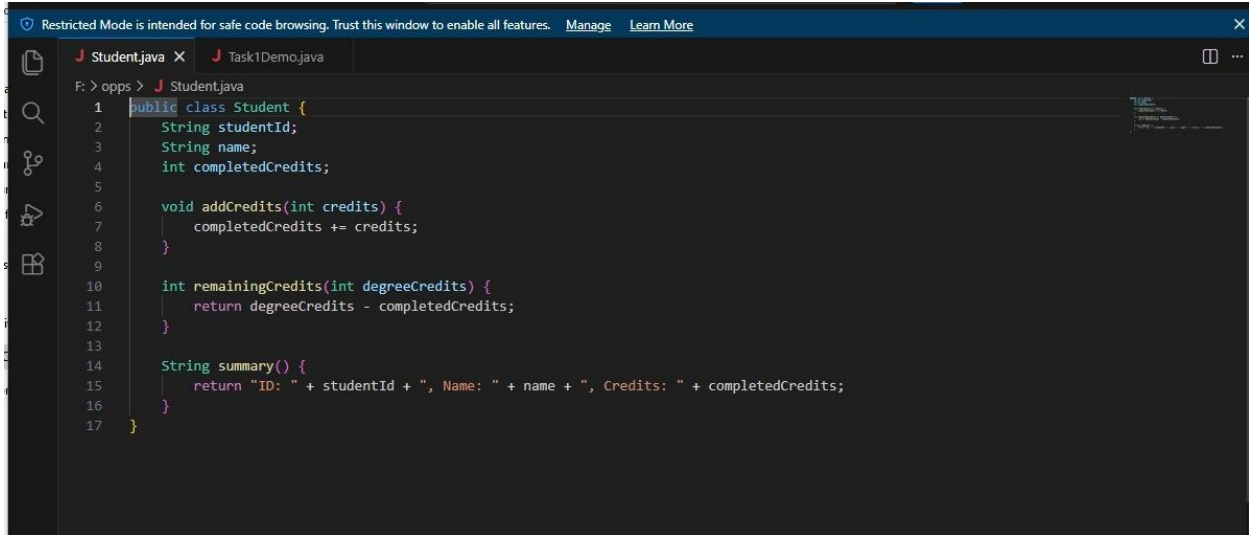
Answer: Because each object allocates its own independent set of instance variables in memory, so modifying `s2` only updates `s2`'s memory state without affecting `s1` or `s3`.

3. Which variables in `Student` represent persistent object state, and why are they instance variables rather than local variables?

Answer: `studentId`, `name`, and `completedCredits` represent persistent state. They are instance variables so they remain saved in memory throughout the object's lifetime, unlike local variables which get destroyed when a method finishes execution.

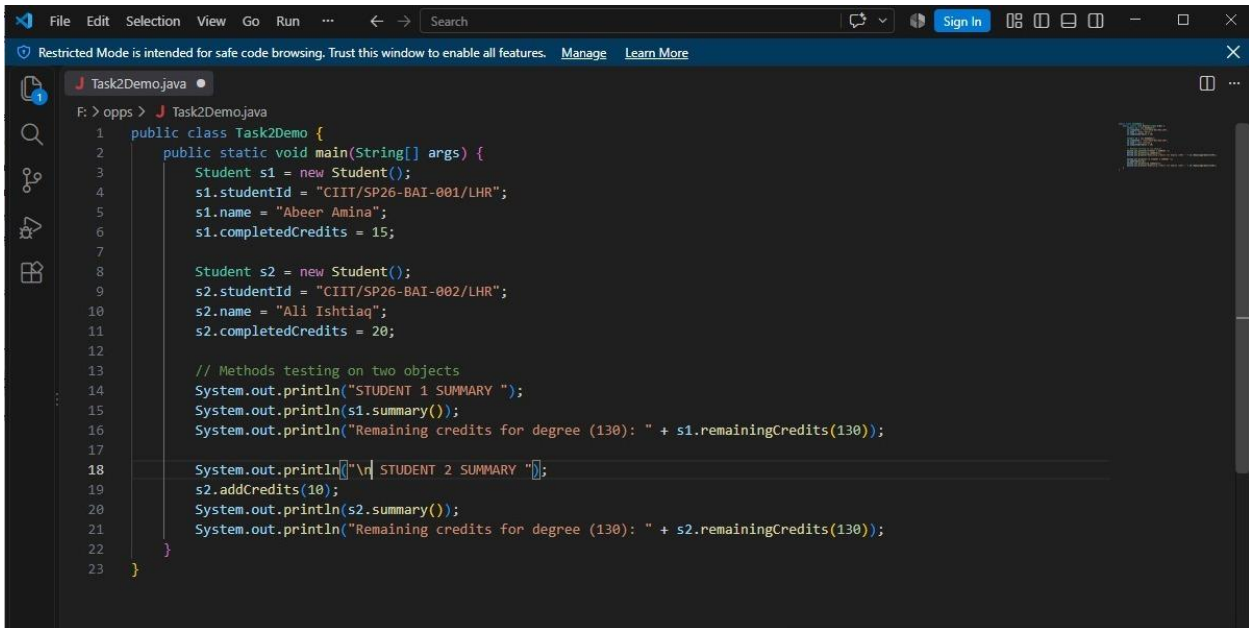
TASK 2: Add Behavior (Methods & Returns)

Step 1: Student.java



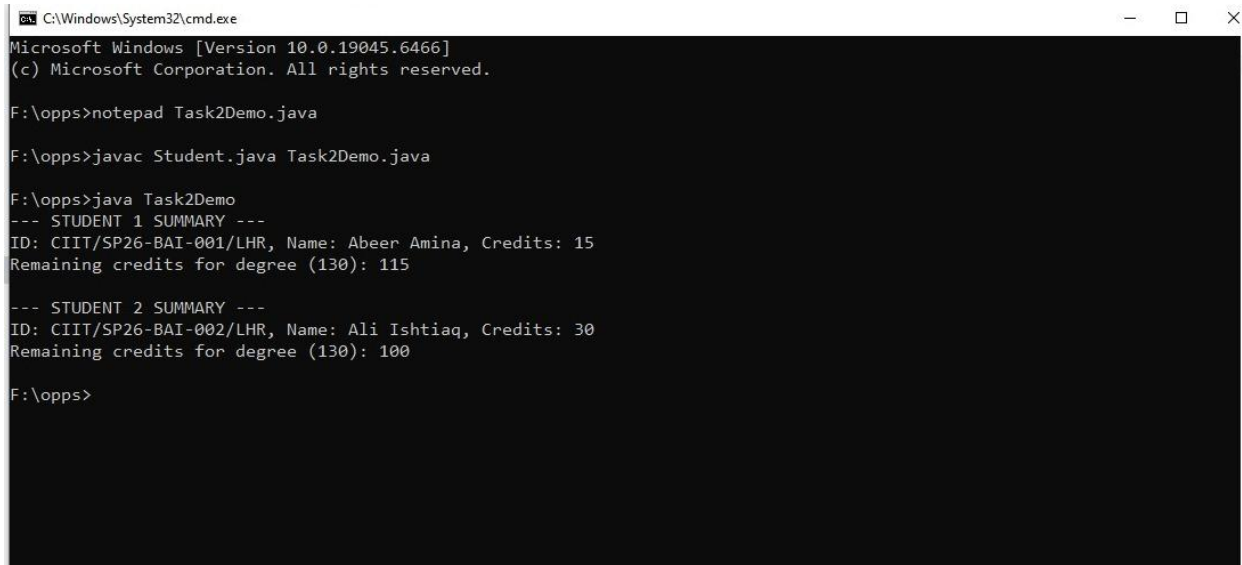
```
1 public class Student {
2     String studentId;
3     String name;
4     int completedCredits;
5
6     void addCredits(int credits) {
7         completedCredits += credits;
8     }
9
10    int remainingCredits(int degreeCredits) {
11        return degreeCredits - completedCredits;
12    }
13
14    String summary() {
15        return "ID: " + studentId + ", Name: " + name + ", Credits: " + completedCredits;
16    }
17 }
```

Step 2: Task2Demo.java



```
1 public class Task2Demo {
2     public static void main(String[] args) {
3         Student s1 = new Student();
4         s1.studentId = "CIIT/SP26-BAI-001/LHR";
5         s1.name = "Abeer Amina";
6         s1.completedCredits = 15;
7
8         Student s2 = new Student();
9         s2.studentId = "CIIT/SP26-BAI-002/LHR";
10        s2.name = "Ali Ishtiaq";
11        s2.completedCredits = 20;
12
13        // Methods testing on two objects
14        System.out.println("STUDENT 1 SUMMARY ");
15        System.out.println(s1.summary());
16        System.out.println("Remaining credits for degree (130): " + s1.remainingCredits(130));
17
18        System.out.println("\n STUDENT 2 SUMMARY ");
19        s2.addCredits(10);
20        System.out.println(s2.summary());
21        System.out.println("Remaining credits for degree (130): " + s2.remainingCredits(130));
22    }
23 }
```

Task 2 : Command Prompt Output



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

F:\opps>notepad Task2Demo.java

F:\opps>javac Student.java Task2Demo.java

F:\opps>java Task2Demo
--- STUDENT 1 SUMMARY ---
ID: CIIT/SP26-BAI-001/LHR, Name: Abeer Amina, Credits: 15
Remaining credits for degree (130): 115

--- STUDENT 2 SUMMARY ---
ID: CIIT/SP26-BAI-002/LHR, Name: Ali Ishtiaq, Credits: 30
Remaining credits for degree (130): 100

F:\opps>
```

Task 2 Concept Check - Answers

1. In addCredits(int credits), what is the difference between the parameter credits and the object field completedCredits?

Answer: credits is a temporary input parameter that receives the value passed during the method call, while completedCredits is an instance field that permanently stores the total accumulated credits of the object in memory.

2. Why is remaining inside remainingCredits(...) better as a local variable than as an instance field?

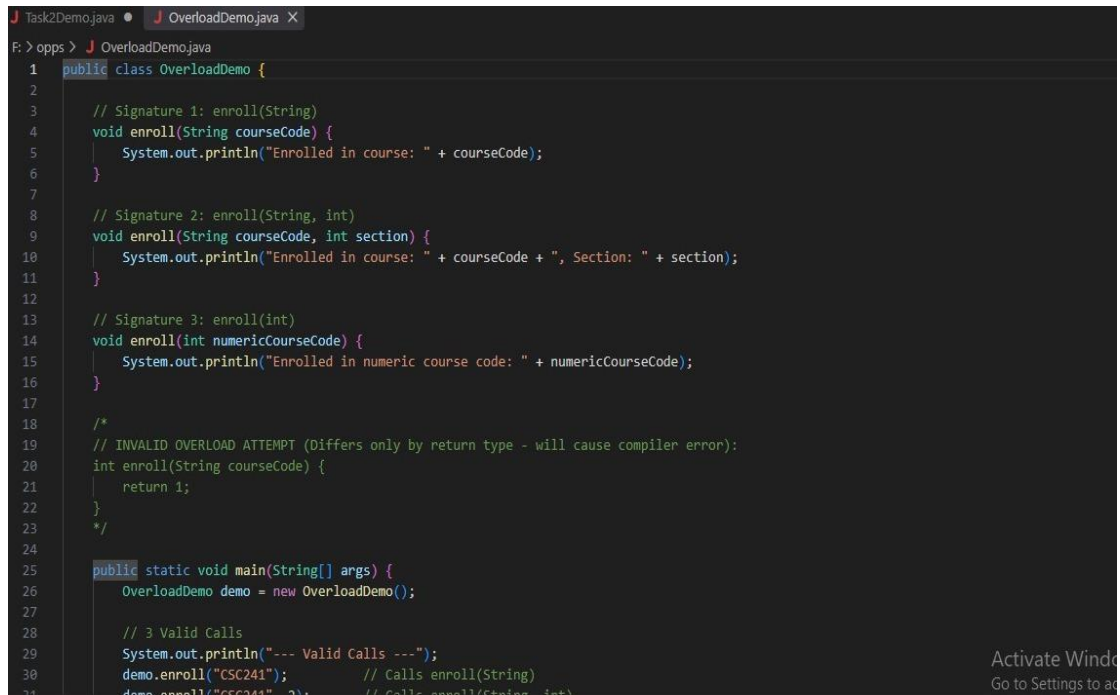
Answer: remaining is a temporary calculation result that is only needed during execution; keeping it as a local variable prevents wasting memory and avoids unnecessarily cluttering the object's permanent state.

3. What information does the return type of remainingCredits(int degreeCredits) give to the caller?

Answer: It informs the caller that executing this method will compute and return a single integer value representing the remaining credits needed.

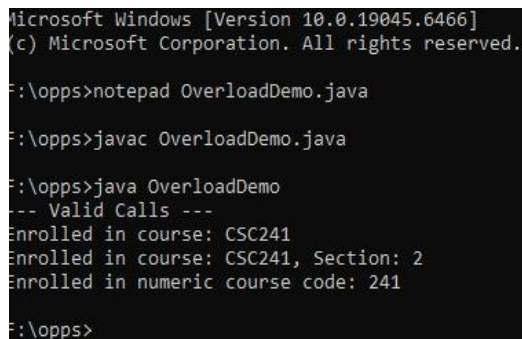
TASK 3: Method Overloading

Step 1: OverloadDemo.java



```
1 public class OverloadDemo {
2
3     // Signature 1: enroll(String)
4     void enroll(String courseCode) {
5         System.out.println("Enrolled in course: " + courseCode);
6     }
7
8     // Signature 2: enroll(String, int)
9     void enroll(String courseCode, int section) {
10        System.out.println("Enrolled in course: " + courseCode + ", Section: " + section);
11    }
12
13    // Signature 3: enroll(int)
14    void enroll(int numericCourseCode) {
15        System.out.println("Enrolled in numeric course code: " + numericCourseCode);
16    }
17
18    /*
19     // INVALID OVERLOAD ATTEMPT (Differs only by return type - will cause compiler error):
20     int enroll(String courseCode) {
21         return 1;
22     }
23    */
24
25    public static void main(String[] args) {
26        OverloadDemo demo = new OverloadDemo();
27
28        // 3 Valid Calls
29        System.out.println("--- Valid Calls ---");
30        demo.enroll("CSC241"); // Calls enroll(String)
31        demo.enroll("CSC241", 2); // Calls enroll(String, int)
```

Step 2 : Command Prompt Output



```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

F:\opps>notepad OverloadDemo.java

F:\opps>javac OverloadDemo.java

F:\opps>java OverloadDemo
--- Valid Calls ---
Enrolled in course: CSC241
Enrolled in course: CSC241, Section: 2
Enrolled in numeric course code: 241

F:\opps>
```

Task 3 Concept Check - Answers

1. What is the method signature of enroll(String courseCode, int section)?

Answer: enroll(String, int) (The signature consists of the method name and its parameter types in order).

2. Why are enroll(String) and enroll(int) valid overloads, while two methods with the same name and parameter types but different return types are not?

Answer: Java uses parameter types and count to differentiate method signatures at compile-time; the return type is not part of the method signature, so it cannot be used to resolve method calls.

3. For the call enroll("CSC241", 2), which overload should the compiler select, and what information does it use to make that decision?

Answer: The compiler selects enroll(String courseCode, int section) because the arguments passed match the parameter types (String, int) in count and order.

TASK 4: Prove Java Pass-by-Value

Step 1: PassByValueDemo.java

```
PassByValueDemo - Notepad
File Edit Format View Help
public class PassByValueDemo {

    // Helper method for Experiment A
    public static void changeNumber(int x) {
        System.out.println(" [Inside changeNumber] Before assignment: x = " + x);
        x = 99;
        System.out.println(" [Inside changeNumber] After assignment: x = " + x);
    }

    // Helper method for Experiment B
    public static void changeStudent(Student st) {
        System.out.println(" [Inside changeStudent] Before modification: credits = " + st.completedCredits);
        st.completedCredits = 99;
        System.out.println(" [Inside changeStudent] After modification: credits = " + st.completedCredits);
    }

    // Helper method for Experiment C
    public static void replaceStudent(Student st) {
        System.out.println(" [Inside replaceStudent] Before reassignment: name = " + st.name);
        st = new Student();
        st.name = "Temporary";
        System.out.println(" [Inside replaceStudent] After reassignment: name = " + st.name);
    }

    public static void main(String[] args) {
        // Experiment A: Primitive Pass-by-Value
        System.out.println("Experiment A: Primitive Variable ");
        int num = 10;
        System.out.println("Before method call: num = " + num);
        changeNumber(num);
        System.out.println("After method call: num = " + num);

        // Experiment B: Object Reference Mutation
        System.out.println("\nExperiment B: Object Reference Mutation");
        Student s = new Student();
        s.studentId = "CIIT/SP26-BAI-003/LHR";
    }
}
```

Step 2: Command Prompt Output

```
Microsoft Windows [Version 10.0.17134.1]
(c) Microsoft Corporation. All rights reserved.

F:\opps>notepad PassByValueDemo.java

F:\opps>javac Student.java PassByValueDemo.java

F:\opps>java PassByValueDemo
Experiment A: Primitive Variable
Before method call: num = 10
    [Inside changeNumber] Before assignment: x = 10
    [Inside changeNumber] After assignment: x = 99
After method call: num = 10

Experiment B: Object Reference Mutation
Before method call: credits = 12
    [Inside changeStudent] Before modification: credits = 12
    [Inside changeStudent] After modification: credits = 99
After method call: credits = 99

Experiment C: Object Reference Reassignment
Before method call: name = Abdul Rehman Azam
    [Inside replaceStudent] Before reassignment: name = Abdul Rehman Azam
    [Inside replaceStudent] After reassignment: name = Temporary
After method call: name = Abdul Rehman Azam

F:\opps>
```

Task 4 Concept Check - Answers

1. **After `changeNumber(int x)` assigns `x = 99`, why does the caller's original `int` variable keep its old value?**

Answer: Because the argument value is copied into the parameter, so `x` is a separate local copy in memory; changing `x` does not affect the original caller variable.

2. **If a `Student` reference is passed to a method, why can changing `st.completedCredits` be visible to the caller?**

Answer: Because the argument value is copied into the parameter as a copy of the object reference pointing to the same heap memory object; thus, modifying `st.completedCredits` mutates the original shared object.

3. **Why does assigning `st = new Student()` inside a method not make the caller's variable refer to the new `Student` object?**

Answer: Because the argument value is copied into the parameter, reassigning `st` only points the local parameter copy to a new object, leaving the caller's original reference variable untouched.

TASK 5: Integrated BAI Challenge (AIExperiment)

Step 1: AIExperiment.java

```
*AIExperiment - Notepad
File Edit Format View Help
public class AIExperiment {
    String experimentName;
    int completedEpochs;
    int targetEpochs;

    // Overload 1: Basic run
    void runEpochs(int epochs) {
        completedEpochs += epochs;
    }

    void runEpochs(int epochs, int bonusEpochs) {
        completedEpochs += (epochs + bonusEpochs);
    }

    int remainingEpochs() {
        int remaining = targetEpochs - completedEpochs;
        return remaining;
    }

    String status() {
        return "Experiment: " + experimentName + " | Completed: " + completedEpochs + "/" + targetEpochs;
    }
}
```

Step 2: Task5Demo.java

```
*Task5Demo - Notepad
File Edit Format View Help
public static void modifyExperiment(AIExperiment exp) {
    exp.completedEpochs += 5;
}

public static void main(String[] args) {
    // Two independent objects
    AIExperiment exp1 = new AIExperiment();
    exp1.experimentName = "CNN Image Classifier";
    exp1.completedEpochs = 10;
    exp1.targetEpochs = 50;

    AIExperiment exp2 = new AIExperiment();
    exp2.experimentName = "LLM Fine-tuning";
    exp2.completedEpochs = 20;
    exp2.targetEpochs = 100;

    System.out.println("INITIAL STATE");
    System.out.println(exp1.status());
    System.out.println(exp2.status());

    // Overloaded methods
    exp1.runEpochs(10);
    exp2.runEpochs(15, 5);

    System.out.println("\n AFTER METHOD CALLS ");
    System.out.println(exp1.status() + " | Remaining: " + exp1.remainingEpochs());
    System.out.println(exp2.status() + " | Remaining: " + exp2.remainingEpochs());

    // Pass-by-value mutation experiment
    modifyExperiment(exp1);
    System.out.println("\nAFTER HELPER MUTATION ON EXP1");
    System.out.println(exp1.status());
    System.out.println(exp2.status());
}
}
```

Step 3: Command Prompt Output

```
Microsoft Windows [Version 10.0.19041.1000]
(c) Microsoft Corporation. All rights reserved.

F:\opps>notepad AIExperiment.java

F:\opps>notepad Task5Demo.java

F:\opps>javac AIExperiment.java Task5Demo.java

F:\opps>java Task5Demo
Error: Could not find or load main class Task5Demo
Caused by: java.lang.ClassNotFoundException: Task5Demo

F:\opps>javac AIExperiment.java Task5Demo.java
Task5Demo.java:3: error: cannot access AIExperiment
    public static void modifyExperiment(AIExperiment exp) {
                                       ^
    bad source file: .\AIExperiment.java
      file does not contain class AIExperiment
    Please remove or make sure it appears in the correct subdirectory of the sourcepath.
1 error

F:\opps>java Task5Demo
Error: Could not find or load main class Task5Demo
Caused by: java.lang.ClassNotFoundException: Task5Demo

F:\opps>javac AIExperiment.java Task5Demo.java

F:\opps>java Task5Demo
INITIAL STATE
Experiment: CNN Image Classifier | Completed: 10/50
Experiment: LLM Fine-tuning | Completed: 20/100

AFTER METHOD CALLS
Experiment: CNN Image Classifier | Completed: 20/50 | Remaining: 30
Experiment: LLM Fine-tuning | Completed: 40/100 | Remaining: 60

AFTER HELPER MUTATION ON EXP1
Experiment: CNN Image Classifier | Completed: 25/50
Experiment: LLM Fine-tuning | Completed: 40/100
```

Task 5 Concept Check - Answers

1. In AIExperiment, which data should be stored as instance variables, and which calculations should remain local to methods?

Answer: experimentName, completedEpochs, and targetEpochs should be instance variables to store state, while dynamic calculations like remaining in remainingEpochs() should remain local variables.

2. Identify one method in your AIExperiment class that changes object state and one method that only computes or returns information.

Answer: runEpochs(int epochs) changes object state by modifying completedEpochs, whereas remainingEpochs() only computes and returns information without altering state.

3. **If you created overloaded methods in this task, state their signatures and explain why the compiler can distinguish them.**

Answer: The signatures are `runEpochs(int)` and `runEpochs(int, int)`. The compiler distinguishes them at compile time based on the difference in parameter count.

Five-Minute Exit Check

1. **What is the difference between a class and an object?**

Answer: A **class** is a blueprint/template that defines attributes and behaviors. An **object** is an instance of a class created in memory with specific state values.

2. **Why does changing `s1.completedCredits` not change `s2.completedCredits`?**

Answer: Because `s1` and `s2` are independent objects with their own separate memory heap locations and instance variables.

3. **What is the method signature of `enroll(String, int)`?**

Answer: The method signature consists of the method name and parameter types:
`enroll(String, int)`.

4. **Why can two methods not be overloaded by return type alone?**

Answer: Because Java compiler resolves method calls based on the method name and argument types, not the return type.

5. **Why can a method mutate an object through a copied reference value but cannot reassign the caller's reference variable?**

Answer: Java uses **pass-by-value**. The copied reference points to the same underlying object in the heap, allowing modification of its properties (mutation), but reassigning the copy inside the method only changes the local copy, leaving the original reference variable unchanged.